ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Faculty of Information Technology, Department of Software Engineering

MID EXAM, October 2025-2026/2 - Database Development with PL/SQL (INSY 8211?)

Instructor: Elie Kayitare - Groups E, F - Duration: 2 hours - Total: 30 marks

Complete questions and model answers for Part A, Part B and Part C. Questions were transcribed from a screen recording, so a few words may be approximate.

General instructions

- Write your name and student ID clearly at the top of your answer sheet.
- Part A: Multiple Choice (10 marks). Part B: Open Questions (10 marks). Part C: Programming (10 marks).
- Answer ALL questions in Parts A and B.
- Part C: answer only TWO out of THREE questions. If you answer all three, only the first two answered are graded.
- Code must be syntactically correct. No collaboration or electronic devices allowed.

Database schema reference

EMPLOYEES

Column	Data type	Description
EMPLOYEE_ID	NUMBER	Primary Key
FIRST_NAME	VARCHAR2(20)	Employee first name
LAST_NAME	VARCHAR2(25)	Employee last name (NOT NULL)
EMAIL	VARCHAR2(25)	Email address
HIRE_DATE	DATE	Date employee was hired (NOT NULL)
JOB_ID	VARCHAR2(10)	Foreign Key to JOBS (NOT NULL)
SALARY	NUMBER(8,2)	Monthly salary
COMMISSION_PCT	NUMBER(2,2)	Commission percentage (NULL if none)
MANAGER_ID	NUMBER	Foreign Key to EMPLOYEES

DEPARTMENTS

Column	Data type	Description
DEPARTMENT_ID	NUMBER	Primary Key
DEPARTMENT_NAME	VARCHAR2(30)	Department name (NOT NULL)
MANAGER_ID	NUMBER	Foreign Key to EMPLOYEES
LOCATION_ID	NUMBER	Foreign Key to LOCATIONS

The rest of EMPLOYEES (including DEPARTMENT_ID) and the JOBS and LOCATIONS tables were not visible in the video.

PART A: Multiple Choice Questions (10 marks) - 2 marks each

Q1. What will be the output of the following PL/SQL block?

```
BEGIN
  FOR v_i IN 5..10 LOOP
    IF v_i = 7 THEN
      EXIT;
    END IF;
    DBMS_OUTPUT.PUT_LINE(v_i);
  END LOOP;
END;
/
```

a) 5 6 7
b) 5 6
c) 5 6 8 10
d) 7 9 10

ANSWER: b) 5 6. The loop prints 5 and 6. At 7, EXIT fires before the print, so the loop stops and 7 is never shown.

Q2. Given v_salary = 12500, what will v_grade be after the following CASE statement executes?

```
CASE
  WHEN v_salary >= 20000 THEN v_grade := 'Executive';
  WHEN v_salary >= 15000 THEN v_grade := 'Senior';
  WHEN v_salary >= 10000 THEN v_grade := 'Mid-Level';
  ELSE v_grade := 'Entry Level';
END CASE;
```

a) 'Executive'
b) 'Senior'
c) 'Mid-Level'
d) 'Entry Level'

ANSWER: c) 'Mid-Level'. CASE tests from the top and stops at the first true branch. 12500 fails the first two tests and passes >= 10000.

Q3. What is the value of v_result after this block executes?

```
DECLARE
  v_commission NUMBER := NULL;
  v_salary     NUMBER := 8000;
  v_result     NUMBER;
BEGIN
  v_result := v_salary + NVL(v_commission, 500);
END;
/
```

a) NULL
b) 8000
c) 8500
d) 500

ANSWER: c) 8500. NVL replaces the NULL commission with 500, so 8000 + 500 = 8500. Without NVL the result would be NULL.

Q4. How many times will DBMS_OUTPUT.PUT_LINE execute in the following block?

```
DECLARE
  v_i NUMBER := 1;
BEGIN
  WHILE v_i <= 4 LOOP
    DBMS_OUTPUT.PUT_LINE(v_i);
    v_i := v_i + 1;
  END LOOP;
END;
/
```

a) 3 times
b) 4 times
c) 5 times
d) It will loop forever

ANSWER: b) 4 times. It prints 1, 2, 3, 4. Then v_i = 5 and the condition fails. It does not loop forever because v_i is incremented.

Q5. Which SQL query will correctly return the department name and total number of employees in each department?

```
-- Query A
```

```
SELECT d.department_name, COUNT(e.employee_id)
FROM departments d, employees e
WHERE d.department_id = e.department_id
GROUP BY d.department_name;

-- Query B
SELECT d.department_name, COUNT(*)
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name;

-- Query C
SELECT d.department_name, COUNT(e.employee_id)
FROM departments d
JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id;
```

a) Query A is correct
b) Query B is correct
c) Query C is correct
d) Both A and B are correct

ANSWER: a) Query A is correct (from the queries as read in the video).

- **A**: the join links each employee to a department and the GROUP BY matches the selected name.
- **B**: runs, but COUNT(*) gives 1 for a department with no employees (the LEFT JOIN row), so those counts are wrong.
- **C**: fails with ORA-00979, because department_name is selected but the GROUP BY uses department_id.

Video was blurry here: double-check Query C against the original paper.

PART B: Open Questions (10 marks) - 2 marks each

Q1. A PL/SQL block has three sections: DECLARE, BEGIN, and EXCEPTION. Which sections are optional and which are required? Briefly explain what each section is used for.

ANSWER:

- **DECLARE** (optional): declares variables, constants, cursors and user-defined types used in the block.
- **BEGIN** (required): holds the executable statements (SQL, loops, IF/CASE, assignments); ends with END;
- **EXCEPTION** (optional): handles runtime errors such as NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE and OTHERS so the program ends cleanly.
- Required: BEGIN ... END. Optional: DECLARE and EXCEPTION.

Q2. Write a SQL query that returns the first name, last name, and department name for all employees. Use a JOIN between the EMPLOYEES and DEPARTMENTS tables. (Hint: join on EMPLOYEES.DEPARTMENT_ID = DEPARTMENTS.DEPARTMENT_ID)

ANSWER:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id;
```

Q3. What is the difference between an IF-ELSIF-ELSE statement and a CASE statement in PL/SQL? Give one situation where you would prefer to use a CASE statement.

ANSWER:

- **IF-ELSIF-ELSE** evaluates a series of independent boolean conditions (ranges, AND/OR, different columns) and runs the first branch that is true.
- **CASE** compares one expression against several values (simple CASE) or evaluates searched conditions; it can also be used as an expression that returns a value.
- Prefer CASE when choosing between many fixed values of one variable, e.g. turning a job code or department id into a label. It is shorter and easier to read than a long ELSIF chain.

- A CASE statement with no matching branch and no ELSE raises CASE_NOT_FOUND; an IF simply does nothing.

Q4. Write a SQL query that returns each employee's first name, last name, and job title. Join the EMPLOYEES table with the JOBS table. Only show employees whose salary is greater than 10000. (Hint: join on EMPLOYEES.JOB_ID = JOBS.JOB_ID)

ANSWER:

```
SELECT e.first_name, e.last_name, j.job_title
FROM employees e
JOIN jobs j
  ON e.job_id = j.job_id
WHERE e.salary > 10000;
```

Q5. What does the NVL function do? Write an example expression using the COMMISSION_PCT column from the EMPLOYEES table that calculates the total pay (salary + commission amount). Handle the case where COMMISSION_PCT may be NULL.

ANSWER: NVL(expr1, expr2) returns expr2 when expr1 is NULL, otherwise it returns expr1. It prevents NULL results in calculations.

```
SELECT employee_id,
       salary + salary * NVL(commission_pct, 0) AS total_pay
FROM employees;
```

If COMMISSION_PCT is NULL, NVL turns it into 0, so total pay equals the salary instead of NULL.

PART C: Programming Questions (10 marks) - choose any 2, 5 marks each

Q1. Write a PL/SQL block that calculates tax for an employee based on their salary category.

Tasks:

- 1. Prompt the user to enter an employee ID (use a variable, e.g. v_emp_id := 100).
- 2. Retrieve the employee's first name, last name, and salary from the EMPLOYEES table.
- 3. Categorize the salary and assign the tax rate:
- Salary < $5,000: Category 'A' - 5% tax
- Salary $5,000 to $10,000: Category 'B' - 10% tax
- Salary $10,001 to $15,000: Category 'C' - 15% tax
- Salary > $15,000: Category 'D' - 20% tax
- 4. Calculate the tax amount and the net salary (salary after deducting tax).
- 5. Display employee name, salary, category, tax amount, and net salary.
- 6. Include appropriate error handling.

Expected output example: banner 'EMPLOYEE TAX CALCULATION', then Employee, Gross Salary, Tax Category, Tax Rate, Tax Amount, Net Salary (e.g. gross $4,800.00, category A, 5%, tax $240.00, net $4,560.00).

ANSWER:

```
SET SERVEROUTPUT ON;
DECLARE
  v_emp_id     employees.employee_id%TYPE := 100;
  v_first_name employees.first_name%TYPE;
  v_last_name  employees.last_name%TYPE;
  v_salary     employees.salary%TYPE;
  v_category   CHAR(1);
  v_tax_rate   NUMBER;
  v_tax_amount NUMBER(10,2);
  v_net_salary NUMBER(10,2);
BEGIN
  SELECT first_name, last_name, salary
  INTO   v_first_name, v_last_name, v_salary
  FROM   employees
  WHERE  employee_id = v_emp_id;

  IF v_salary < 5000 THEN
    v_category := 'A'; v_tax_rate := 0.05;
  ELSIF v_salary <= 10000 THEN
    v_category := 'B'; v_tax_rate := 0.10;
  ELSIF v_salary <= 15000 THEN
    v_category := 'C'; v_tax_rate := 0.15;
  ELSE
    v_category := 'D'; v_tax_rate := 0.20;
  END IF;

  v_tax_amount := v_salary * v_tax_rate;
  v_net_salary := v_salary - v_tax_amount;

  DBMS_OUTPUT.PUT_LINE('===== EMPLOYEE TAX CALCULATION =====');
  DBMS_OUTPUT.PUT_LINE('Employee    : ' || v_first_name || ' ' || v_last_name);
  DBMS_OUTPUT.PUT_LINE('Gross Salary: $' || TO_CHAR(v_salary, 'FM999,990.00'));
  DBMS_OUTPUT.PUT_LINE('Tax Category: ' || v_category);
  DBMS_OUTPUT.PUT_LINE('Tax Rate    : ' || v_tax_rate * 100 || '%');
  DBMS_OUTPUT.PUT_LINE('Tax Amount  : $' || TO_CHAR(v_tax_amount, 'FM999,990.00'));
  DBMS_OUTPUT.PUT_LINE('Net Salary  : $' || TO_CHAR(v_net_salary, 'FM999,990.00'));
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: no employee with ID ' || v_emp_id);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
```

Remember: %TYPE, SELECT ... INTO (exactly one row), IF-ELSIF for ranges, handle NO_DATA_FOUND. Boundaries: 5,000 to 10,000 is B; 10,001 to 15,000 is C.

Q2. Write a PL/SQL block that performs the following tasks using loops.

Requirements:

- 1. Use a FOR loop to display all numbers from 1 to 10, but SKIP the number 5.
- 2. Use a WHILE loop to calculate and display the salary of an employee after annual raises, starting from a salary of 5000, applying a 10% raise each year, until the salary reaches or exceeds 8000. Display the salary after each raise and the number of years it took.
- 3. Use a FOR loop in REVERSE to print a countdown from 5 down to 1, then display 'Done!'
- Each of the three tasks should be clearly separated with a DBMS_OUTPUT.PUT_LINE divider line (e.g. a line of '=' characters).

ANSWER:

```
SET SERVEROUTPUT ON;
DECLARE
  v_salary NUMBER := 5000;
  v_years  NUMBER := 0;
BEGIN
  -- Task 1: FOR loop 1..10, skip 5
  DBMS_OUTPUT.PUT_LINE('Task 1: numbers 1 to 10 (skip 5)');
  FOR i IN 1..10 LOOP
    IF i = 5 THEN
      CONTINUE;
    END IF;
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('==============================');

  -- Task 2: WHILE loop, 10% raise until salary >= 8000
  DBMS_OUTPUT.PUT_LINE('Task 2: salary raises');
  WHILE v_salary < 8000 LOOP
    v_salary := v_salary * 1.10;
    v_years  := v_years + 1;
    DBMS_OUTPUT.PUT_LINE('Year ' || v_years || ': ' || ROUND(v_salary, 2));
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Years needed: ' || v_years);
  DBMS_OUTPUT.PUT_LINE('==============================');

  -- Task 3: REVERSE countdown
  DBMS_OUTPUT.PUT_LINE('Task 3: countdown');
  FOR i IN REVERSE 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Done!');
END;
/
```

*Expected: Task 1 prints 1 2 3 4 6 7 8 9 10. Task 2 gives 5500, 6050, 6655, 7320.5, 8052.55 and takes **5 years**. Task 3 prints 5 4 3 2 1 then Done! CONTINUE needs Oracle 11g or later (otherwise use IF i <> 5). In REVERSE the range is still written 1..5.*

Q3. Write a PL/SQL block that uses a cursor FOR loop to display a report of employees and their departments.

Requirements:

- 1. Use a cursor FOR loop (FOR ... IN (SELECT ...) LOOP) to loop through all rows in the DEPARTMENTS table.
- 2. For each department, print the department name as a section header.
- 3. Inside the outer loop, use a second (nested) cursor FOR loop to find and print all employees in that department. Display each employee's first name, last name and ... (rest cut off in the video).

ANSWER:

```
SET SERVEROUTPUT ON;
BEGIN
  FOR dept IN (SELECT department_id, department_name
               FROM departments
               ORDER BY department_name) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('=== Department: ' || dept.department_name || ' ===');

    FOR emp IN (SELECT first_name, last_name, salary
                FROM employees
                WHERE department_id = dept.department_id
                ORDER BY last_name) LOOP
      DBMS_OUTPUT.PUT_LINE('   ' || emp.first_name || ' ' || emp.last_name
                           || ' - ' || emp.salary);
    END LOOP;

  END LOOP;
END;
/
```

Remember: a cursor FOR loop opens, fetches and closes the cursor automatically. The inner query uses the outer record (dept.department_id). Add whichever extra columns the real requirement asks for (salary was my guess).